SafeRoute Guardian — Simple Build Checklist (for Antigravity)

How to use this:

1. Do one step at a time, in order.
2. For each step, ask ChatGPT for a prompt using the "Tell ChatGPT" line, then paste that prompt into Antigravity.
3. Run the app and do the "Check it works" test **before** moving on.
4. When a step works, save it (git commit) so you can always go back.

Each step says: what we build, why, what to tell ChatGPT, and how to check it.

Paste this at the start of every ChatGPT chat (project brief)

> We are building "SafeRoute Guardian", a web app for a hackathon. The user enters a start point, an end point and a time. The app shows the fastest route, checks it against danger zones (red = high risk, yellow = moderate, green = lower) and shows a safer route that takes only a few minutes longer. Each route gets a Safety Score out of 100. Rule: Route Cost = Travel Time + Risk Penalty. Extra features: one-tap "Switch to safer route", an AI safety assistant (Gemini), and a fake SOS simulation. Demo city: Bengaluru, with made-up (synthetic) risk zones. Tech: React + Tailwind CSS for the website, Mapbox for the map and routes, Turf.js for map calculations, Node.js + Express for the server, Gemini API for the AI. I am building with Antigravity. Give me one clear prompt I can paste into Antigravity for the step below. Keep it simple and tell it to only do this step.

Part A — Get ready (do this first)

Step 1 — Accounts and keys

- **What:** Make the accounts you need and collect the keys.
- **Why:** The map, routes and AI will not work without them.
- **Do it yourself (no prompt needed):**
  - Mapbox account → copy the public token (starts with `pk.`)
  - Google AI Studio → create a Gemini API key
  - GitHub repo for the team
- **Rule:** never paste keys into ChatGPT or into chat prompts. Keys go only in a private settings file (`.env`) that the project reads.
- **Check it works:** every teammate has access to the repo, and keys are saved somewhere private.

Step 2 — Decide the demo story

- **What:** Pick ONE trip that we will show to judges: a start, an end and a night-time hour in Bengaluru where the fastest road passes a danger zone.
- **Why:** The whole project is built to make this one demo look great.
- **Do it yourself:** write the story in one sentence, e.g. "From A to B at 10:30 PM, fastest = 18 min but risky, safer = 21 min."
- **Check it works:** everyone on the team can say the story in one sentence.

Step 3 — Create the empty project

- **What:** A starter project with a website part (client) and a server part (server).
- **Why:** Everything else sits inside it.
- **Tell ChatGPT:** "Write an Antigravity prompt to create a React + Tailwind CSS website folder called `client` and a Node.js + Express server folder called `server`, with a settings file for secret keys, and a simple health check page on the server."
- **Check it works:** the website opens in the browser, and opening the server's health page shows "ok".

Part B — The map and the data

Step 4 — Dark map of Bengaluru

- **What:** A full-screen dark map centred on Bengaluru.
- **Why:** This is the main screen of the app.
- **Tell ChatGPT:** "Antigravity prompt: show a dark Mapbox map of Bengaluru that fills the screen, with zoom buttons. The Mapbox token comes from the settings file."
- **Check it works:** you can drag and zoom the map.

Step 5 — Danger zones data

- **What:** A file with made-up danger zones (red, yellow, green shapes) over Bengaluru.
- **Why:** The app needs something to check routes against.
- **How to make the shapes (easy way):** draw them at geojson.io and save the file. Put one **red** zone right on the fastest road of your demo trip, some yellow and green ones elsewhere, and keep the safer road clear of red.
- **Tell ChatGPT:** "I have a zones file with red/yellow/green shapes. Write an Antigravity prompt that checks the file is valid and loads it into the project."
- **Check it works:** zones are in the right place on the map (not in the ocean!).

Step 6 — Show zones on the map

- **What:** Coloured zones drawn on the map, a legend, and a button to hide/show them. Clicking a zone shows its name and risk level.
- **Why:** Judges can instantly see where the danger is.

- **Tell ChatGPT:** "Antigravity prompt: draw the zones from my file on the Mapbox map — red, yellow, green with soft transparency — add a legend, a show/hide button, and a popup when a zone is clicked."
- **Check it works:** colours match the legend, the button hides them, the popup shows details.

Part C — Taking the user's trip

Step 7 — Search boxes and time

- **What:** A side panel with "From" and "To" search boxes (with suggestions), a time picker, quick time buttons (Now, 2 PM, 9 PM, 11 PM), and a "Find safer route" button.
- **Why:** This is how the user tells the app about the trip.
- **Tell ChatGPT:** "Antigravity prompt: add a left side panel with From and To place search using Mapbox, a date-time picker with quick buttons, markers on the map for both places, and a Find safer route button that is disabled until both places are chosen."
- **Check it works:** you can pick two places, see two markers and change the time.

Step 8 — "Load demo trip" button

- **What:** One button that fills in your demo trip automatically.
- **Why:** On stage, you don't want to type. This is your safety net.
- **Tell ChatGPT:** "Antigravity prompt: add a Load demo trip button that fills From, To and time with these values: [paste your demo start, end, time]."
- **Check it works:** one click fills everything.

Step 9 — Get real routes

- **What:** When the button is pressed, get the fastest road route(s) between the two places and draw them on the map.
- **Why:** These are the routes we will check for danger.
- **Tell ChatGPT:** "Antigravity prompt: when Find safer route is pressed, call the Mapbox Directions API for the route between From and To (with alternatives), draw each route on the map, and show its minutes and distance."
- **Check it works:** the demo trip draws a road route with a sensible time (around 18 minutes).

Part D — The brain (the most important part)

Step 10 — Check how much of a route is in danger zones

- **What:** For each route, work out how many kilometres pass through red, yellow and green zones.
- **Why:** This is the "risk check" that makes our app different from normal maps.

- **Tell ChatGPT:** "Antigravity prompt: using Turf.js, write a function that takes a route and my zones file and returns how many km of the route are inside red, yellow and green zones, and which zones it crosses. If zones overlap, the higher risk wins. Also split the route into small pieces so each piece knows its risk colour."
- **Check it works:** the demo's fastest route shows some red kilometres; a route far from the zones shows none.

Step 11 — Safety Score and the time-of-day effect

- **What:** Turn the danger distance into a Safety Score (0–100) and a risk penalty, and make night count more than day.
- **Why:** This is our headline idea: Route Cost = Travel Time + Risk Penalty, and time matters.
- **Tell ChatGPT:** "Antigravity prompt: create a scoring module. Red, yellow, green zones have weights 3, 1.5, 0.5. Night (9 PM–6 AM) multiplies risk by 1.6, evening (5–9 PM) by 1.3, day by 1.0. Compute a Safety Score from 0 to 100 (more danger = lower score), a risk penalty in minutes, and Route Cost = travel time + penalty. Put all the numbers in one settings file so we can tune them. Add simple tests."
- **Check it works:** same route scores lower at 11 PM than at 2 PM. More red km means a lower score.

Step 12 — Find the safer route

- **What:** Create extra detour routes that go around red zones, score them all, and pick the best (lowest cost) as the "Safe corridor".
- **Why:** The app must not just warn — it must offer a better route.
- **Tell ChatGPT:** "Antigravity prompt: find red zones on the fastest route, create a few detour waypoints around them, request routes through those waypoints, remove duplicates and routes that are too long, score all routes and choose the lowest-cost one as the Safe corridor. Show the safer route only if its score is at least 10 points better. If none exists, show a friendly message."
- **Check it works:** the demo trip gives a safer route with a slightly longer time and a clearly higher score.
- **Important:** if the numbers aren't close to 18 min / 58 and 21 min / 94, change the **zone shapes and tuning numbers** (don't type the numbers in by hand).

Part E — The screen judges will see

Step 13 — Show the results nicely

- **What:** Two cards: **Fastest** and **Safe corridor**, each with time, distance, Safety Score (big, coloured) and zones crossed. A line like "+3 min for +36 safety points". On the map, the fastest route shows red pieces where it is risky; the safe route is a clean green line.

- **Why:** This is the "wow" moment of the demo.
- **Tell ChatGPT:** "Antigravity prompt: after analysis, show two result cards (Fastest and Safe corridor) with time, distance, Safety Score coloured red/yellow/green, and the trade-off line. Colour the fastest route on the map by risk piece, draw the safe route in green, zoom to fit both, and add loading and error messages."
- **Check it works:** demo trip shows both cards and the colours on the map.

Step 14 — One-tap "Switch to safer route"

- **What:** A big button that makes the safer route the active one, instantly, with an Undo.
- **Why:** The pitch says "actionable, not just a warning".
- **Tell ChatGPT:** "Antigravity prompt: add a Switch to safer route button that instantly makes the safe route the highlighted active route on the map and cards, without fetching anything again, plus an Undo."
- **Check it works:** one click changes the highlight, Undo brings it back.

Step 15 — Show why time matters

- **What:** A small label such as "Late night — risk ×1.6", and re-running with a different time updates the scores.
- **Why:** Shows judges the "right now" idea from the problem statement.
- **Tell ChatGPT:** "Antigravity prompt: show the current time band and its risk multiplier in the results, and make sure changing the time and pressing Find safer route updates the scores."
- **Check it works:** switch from 2 PM to 11 PM and the scores change.

Part F — The extras

Step 16 — AI safety assistant (Gemini)

- **What:** A small chat box where users ask things like "Why is the fastest route risky?" or "Is +3 minutes worth it?". The server talks to Gemini, so the key stays hidden.
- **Why:** It is one of our listed key features.
- **Tell ChatGPT:** "Antigravity prompt: add a server endpoint that sends the user's question plus a small summary of the two routes to Gemini and returns a short answer. Rules for the AI: use only the given data, never invent incidents, say 'safer' not 'safe', say the data is demo data, keep answers to 4 short sentences. Add a chat panel on the website with 3 suggested questions. If Gemini fails, show a simple built-in explanation instead."
- **Check it works:** 3 sample questions get sensible answers; off-topic questions are politely declined; turning off internet still shows the built-in explanation.

Step 17 — SOS simulation

- **What:** An SOS button that plays a fake emergency flow: countdown → alert sent → contacts notified → location shared → help dispatched. Clearly labelled "SIMULATION".
- **Why:** Another listed feature — and it must never contact anyone for real.
- **Tell ChatGPT:** "Antigravity prompt: add a floating SOS button that opens a window labelled 'SIMULATION — no real alerts are sent', with a 5-second cancellable countdown and a timeline of simulated steps. It must not send any real messages or calls."
- **Check it works:** the flow plays and can be cancelled; nothing is sent anywhere.

Part G — Make it safe for the demo

Step 18 — Backup mode

- **What:** Save the demo's routes so the app still works if the internet or an API fails. Add a `?demo=1` mode that uses saved routes.
- **Why:** Demos fail at the worst time. This is your insurance.
- **Tell ChatGPT:** "Antigravity prompt: after a successful demo run, save the routes for reuse. Add a demo mode (?demo=1) that uses saved routes instead of calling Mapbox, while still running our risk checking. Add timeouts and friendly error messages."
- **Check it works:** with `?demo=1`, the demo works without calling the routing API.

Step 19 — Polish and phone check

- **What:** Make it look good and work on phones: same dark + neon-green style as the pitch deck, big buttons, readable text, a footer note "Demo data is synthetic. Scores are not a guarantee of safety."
- **Why:** First impressions, and judges often try it on a phone.
- **Tell ChatGPT:** "Antigravity prompt: make the whole app responsive for phones, tablets and desktops, with a dark theme and neon-green accents, large tap targets, text labels along with colours, and a permanent note that demo data is synthetic."
- **Check it works:** open it on a phone — nothing is cut off and buttons are easy to tap.

Step 20 — Test everything

Go through this list and tick each line:

- [ ] Demo trip at 10:30 PM: fastest is risky, safer route scores higher
- [ ] Same trip at 2 PM: scores are different
- [ ] A trip with no danger zones: friendly "already fine" message
- [ ] Same start and end: error message
- [ ] A place outside Bengaluru: error message
- [ ] Switch to safer route and Undo both work

- [ ] Show/hide zones works
- [ ] Assistant answers 3 questions correctly; works offline with the built-in answer
- [ ] SOS plays, cancels, and sends nothing
- [ ] Works on a phone
- [ ] No secret keys are visible in the website's code (ask Antigravity to check)

Part H — Go live and present

Step 21 — Put it online

- **What:** Publish the website (Vercel or Netlify) and the server (Render or Railway).
- **Tell ChatGPT:** "Give me step-by-step instructions to deploy a React (Vite) client to Vercel and a Node/Express server to Render, including which environment variables to set, how to connect them, and how to restrict my Mapbox token to my website address."
- **Check it works:** the public link works from a phone on mobile data. (Free servers can be slow to wake up, so open it 5 minutes before presenting.)

Step 22 — Prepare the pitch

- **Make a 3-minute demo script:** problem → load demo trip → show risky fastest route → show safer route (+3 min) → change time → switch route → ask the assistant → SOS simulation → tagline.
- **Record a backup video** of the full demo.
- **Prepare answers** to likely questions: Where does the data come from? (made-up for the demo; real version would use public data and community reports) · Why not Google Maps? (it optimises time; we add safety to the route cost) · Is SOS real? (no, simulated on purpose) · What about privacy? (no location is stored).
- **Practise 3 times** with a timer.

If time is short

Must have: Steps 3–14 (map, zones, trip, routes, risk check, score, safer route, result cards, switch button), Step 8 (demo button), Step 18 (backup mode). **Nice to have:** Steps 16 (assistant), 17 (SOS), 15 (time label). **Cut first:** animations, walking mode, extra polish.

Tips for working with Antigravity + ChatGPT

- **One step per prompt.** Big "build everything" prompts break things.
- **Always say what to keep.** Add: "Do not change anything unrelated to this step."
- **If something breaks,** copy the exact error message and ask ChatGPT for a fix prompt. Don't guess.
- **Save after every working step** (git commit). If the next step breaks, go back.
- **Check each step in the browser** before asking for the next one.

- **Never paste secret keys** into ChatGPT, Antigravity prompts or screenshots.
- **Keep the numbers in one settings file** (weights, time multipliers, thresholds) so you can tune the demo easily.
- **Be honest in the pitch:** the zones are synthetic demo data, and "safer" is not "100% safe".